TEK4090 – Introduction to Modern Control
Assignment 2 v4.0

Due: October 23 2024, 13:15

---

1. (10 points) Consider the continuous-time system,

$$\ddot{x} = u \tag{1}$$

$$J(t_0) = \frac{1}{2}x^2(0) + r\int_0^2 u^2 dt. \tag{2}$$

Hint: read §6.3 in Lewis, Syrmos, & Vraibie for some relevant example problems

(a) Write the Hamilton-Jacobi-Bellman (HJB equation), eliminating $u(t)$

(b) Assume that the optimal cost-to-go function $J^*(t)$ is given by,

$$J^*(t) = \frac{1}{2}s_1(t)x^2(t) + s_2(t)x(t)\dot{x}(t) + \frac{1}{2}s_3(t)\dot{x}^2(t). \tag{3}$$

Use the HJB equation to derive coupled differential equations for these $s_i(t)$'s, and find boundary conditions for these equations.

(c) Write the linear state feedback optimal control policy in terms of the $s_i(t)$'s.

Ans: I assume there is a typo in the performance index. I will therefore use the below instead:

$$J(t_0) = \frac{1}{2}x^2(2) + r\int_0^2 u^2 dt.$$

.

(a) The HJB equation and Hamiltonian is:

$$-\frac{\partial J^*}{\partial t} = -J_t^* = \min_u(H(x, u, J_x^*, t))$$
$$H(x, u, \lambda, t) = L(x, u, t) + \lambda^\top f(x, u, t)$$

I must first find the Hamiltonian, then I can find the HJB equation. To find the Hamiltonian I need to first find $\mathring{x} = f(x, u, t)$. I therefore restate the system dynamics by defining the state $X := (x_1, x_2)^\top$ where $x_1 := x$ and $x_2 = \mathring{x}$. We then get $\mathring{X} = (\mathring{x}_1, \mathring{x}_2)^\top = (x_2, u)^\top$ and we've found our f. The Hamiltonian is therefore: $H(X, u, \lambda, t) = ru^2 + \lambda^\top(x_2, u)^\top$ where λ is a column vector. The HJB equation must therefore be: $-J_t^* = \min_u(ru^2 + (J_X^*)^\top(x_2, u)^\top)$. Let's expand the terms in the HJB equation:

$$J_X^* = \begin{pmatrix} \frac{\partial J^*}{x_1} \\ \frac{\partial J^*}{x_2} \end{pmatrix} = \begin{pmatrix} J_{x_1}^* \\ J_{x_2}^* \end{pmatrix}$$
$$-J_t^* = \min_u(ru^2 + J_{x_1}^* x_2 + J_{x_2}^* u)$$

We want to minimize the expression with respect too u. This u is given where the derivative wrt. u is 0 since J is convex in u assuming $r > 0$.

$$\frac{\partial}{\partial u}(ru^2 + J^*_{x_1}x_2 + J^*_{x_2}u) = 0$$
$$= 2ru + J^*_{x_2} = 0 \implies u^* = -\frac{J^*_{x_2}}{2r}$$

We substitute this u back into the HJB equation and remove the minimization since the u we found does that.

$$-J^*_t = r(-\frac{J^*_{x_2}}{2r})^2 + J^*_{x_1}x_2 + J^*_{x_2}(-\frac{J^*_{x_2}}{2r})$$
$$-J^*_t = J^*_{x_1}x_2 - \frac{(J^*_{x_2})^2}{4r} \quad \blacksquare$$

Which is the HJB equation with u eliminated.

(b) To find the coupled differential equations for the s_i's we want to substitute the partial derivatives of the assumed solution into our HJB equation with u eliminated. Recall that $x = x_1$ and $\mathring{x} = x_2$

$$J^*_t = \frac{\partial}{\partial t}J^* = \frac{1}{2}\mathring{s}_1x_1^2 + s_1x_1\mathring{x}_1 + \mathring{s}_2x_1x_2 + s_2\mathring{x}_1x_2 + s_2x_1\mathring{x}_2 + \frac{1}{2}\mathring{s}_3x_2^2 + s_3x_2\mathring{x}_2$$
$$J^*_{x_1} = \frac{\partial}{\partial x_1}J^* = s_1x_1 + s_2x_2$$
$$J^*_{x_2} = \frac{\partial}{\partial x_2}J^* = s_2x_1 + s_3x_2$$

We substitute these into our HJB equation:

$$-J^*_t = J^*_{x_1}x_2 - \frac{(J^*_{x_2})^2}{4r}$$
$$-(\frac{1}{2}\mathring{s}_1x_1^2 + s_1x_1\mathring{x}_1 + \mathring{s}_2x_1x_2 + s_2\mathring{x}_1x_2 + s_2x_1\mathring{x}_2 + \frac{1}{2}\mathring{s}_3x_2^2 + s_3x_2\mathring{x}_2)$$
$$= (s_1x_1 + s_2x_2)x_2 - \frac{(s_2x_1 + s_3x_2)^2}{4r}$$

We expand the expression and gather all the derivatives of s_i by themselves:

$$\frac{1}{2}\mathring{s}_1x_1^2 + \mathring{s}_2x_1x_2 + \frac{1}{2}\mathring{s}_3x_2^2$$
$$= \frac{1}{4r}(s_2^2x_1^2 + 2s_2x_1s_3x_2 + s_3^2x_2^2) - x_2s_1x_1 - s_2x_2^2 - s_1x_1\mathring{x}_1$$
$$-s_2\mathring{x}_1x_2 - s_2x_1\mathring{x}_2 - s_3x_2\mathring{x}_2$$

Now substitute $\mathring{x}_1 = x_2$ and $\mathring{x}_2 = u = -\frac{J^*_{x_2}}{2r} = \frac{-1}{2r}(s_2x_1 + s_3x_2)$:

$$\frac{1}{2}\mathring{s}_1x_1^2 + \mathring{s}_2x_1x_2 + \frac{1}{2}\mathring{s}_3x_2^2$$
$$= \frac{1}{4r}(s_2^2x_1^2 + 2s_2x_1s_3x_2 + s_3^2x_2^2) - x_2s_1x_1 - s_2x_2^2 - s_1x_1x_2$$
$$-s_2x_2x_2 - s_2x_1(\frac{-1}{2r}(s_2x_1 + s_3x_2)) - s_3x_2(\frac{-1}{2r}(s_2x_1 + s_3x_2))$$

We expand the expression:

$$\frac{1}{2}\mathring{s}_1x_1^2 + \mathring{s}_2x_1x_2 + \frac{1}{2}\mathring{s}_3x_2^2$$
$$= \frac{1}{4r}s_2^2x_1^2 + \frac{1}{2r}s_2s_3x_1x_2 + \frac{1}{4r}s_3^2x_2^2 - s_1x_1x_2 - s_2x_2^2 - s_1x_1x_2$$
$$-s_2x_2^2 + \frac{1}{2r}s_2^2x_1^2 + \frac{1}{2r}s_2s_3x_1x_2 + \frac{1}{2r}s_2s_3x_1x_2 + \frac{1}{2r}s_3^2x_2^2$$

Gather and factorize:

$$\mathring{s}_1(\frac{1}{2}x_1^2) + \mathring{s}_2(x_1x_2) + \mathring{s}_3(\frac{1}{2}x_2^2)$$
$$= \frac{3}{2r}s_2^2(\frac{1}{2}x_1^2) + (\frac{3}{2r}s_2s_3 - 2s_1)(x_1x_2) + (\frac{3}{2r}s_3^2 - 4s_2)(\frac{1}{2}x_2^2)$$

It should now be obvious that the coupled ODEs are:

$$\mathring{s}_1 = \frac{3}{2r}s_2^2$$
$$\mathring{s}_2 = \frac{3}{2r}s_2s_3 - 2s_1$$
$$\mathring{s}_3 = \frac{3}{2r}s_3^2 - 4s_2 \quad \blacksquare$$

To find their boundary conditions we can use the fact that:

$$J(2) = \frac{1}{2}x_1^2(2) = \frac{1}{2}s_1(2)x_1^2(2) + s_2(2)x_1(2)x_2(2) + \frac{1}{2}s_3(2)x_2^2(2)$$

Which states that:

$$s_1(2) = 1, \quad s_2(2) = 0, \quad s_3(2) = 0 \quad \blacksquare$$

I've now provided the coupled ODEs and the their boundary conditions.

(c) I've done so in the previous task so I'll merely state it here:

$$u = -\frac{J^*_{x_2}}{2r} = \frac{-1}{2r}(s_2x_1 + s_3x_2) \quad \blacksquare$$

Which is a linear state-feedback controller in terms of s_i.

2. (10 points) Pick a system from a paper that you have found on a topic that you are interested in, potentially the topic you will choose for your final project.
   (a) What is/are the paper(s) you are drawing from?
   (b) What are the dynamics of the system in question? What are the inputs and outputs?
   (c) What is the system trying to achieve?
   (d) What are the challenges for the system?
   (e) What control tools can help the system overcome the challenges in achieving its goals?

Ans:

(a) Quadcopter UAV Control based on Input-Output Linearization and PID

(b)

(c)

(d)

(e)

3. (20 points) Suppose that $f : \mathbb{R} \mapsto \mathbb{R}$ is convex, and $a < b$ are in the domain of f.
   (a) Show that

$$f(x) \leq \frac{b-x}{b-a} f(a) + \frac{x-a}{b-a} f(b) \tag{4}$$

   for all $x \in [a, b]$
   (b) Show that

$$\frac{f(x)-f(a)}{x-a} \leq \frac{f(b)-f(a)}{b-a} \leq \frac{f(b)-f(x)}{b-x} \tag{5}$$

   for all $x \in (a, b)$. Draw a sketch that illustrates this inequality
   (c) Suppose that f is differentiable. Use the result in (b) to show that

$$f'(a) \leq \frac{f(b)-f(a)}{b-a} \leq f'(b). \tag{6}$$

   (d) Suppose f is twice differentiable. Use the result in (c) to show that $f''(a) \geq 0$ and $f''(b) \geq 0$.

Ans:

(a) Since f is convex, by definition we know that.

$$\forall x, y \in \mathbb{R}, \quad \forall \alpha, \beta \geq 0, \quad \alpha + \beta = 1$$
$$f(\alpha x + \beta y) \leq \alpha f(x) + \beta f(y)$$

We can therefore pick $x, y \in \mathbb{R}$ such that $x = a,\ y = b,\ a < b$ which results in:

$$f(\alpha a + \beta b) \leq \alpha f(a) + \beta f(b)$$

Now set $\alpha := \frac{b-x}{b-a}$ and $\beta := \frac{x-a}{b-a}$. x is here a new variable, unrelated to previous mentions of x. Notice that:

$$x \in [a, b] \implies \alpha \in [0, 1] \wedge\ \beta \in [0, 1]$$
$$\alpha + \beta = \frac{b-x}{b-a} + \frac{x-a}{b-a} = \frac{b-x+x-a}{b-a}$$
$$= \frac{b-a}{b-a} = 1$$

Therefore α and β as defined satisfy their conditions for $x \in [a, b]$. We substitute these variables into our inequality and rewrite.

$$f(\frac{b-x}{b-a}a + \frac{x-a}{b-a}b) \leq \frac{b-x}{b-a}f(a) + \frac{x-a}{b-a}f(b)$$

$$f(\frac{ab-ax+bx-ab}{b-a}) \leq \frac{b-x}{b-a}f(a) + \frac{x-a}{b-a}f(b)$$

$$f(\frac{x(b-a)}{b-a}) \leq \frac{b-x}{b-a}f(a) + \frac{x-a}{b-a}f(b)$$

$$f(x) \leq \frac{b-x}{b-a}f(a) + \frac{x-a}{b-a}f(b) \quad \blacksquare$$

Which is the inequality I was tasked to show.

(b) We know from the previous assignment that:

$$\forall x \in [a, b], \quad a < b, \quad f(x) \leq \frac{b-x}{b-a}f(a) + \frac{x-a}{b-a}f(b)$$

I'll rewrite this inequality twice to first show the left side then the right

side of the inequality in the task:

$$f(x) \leq \frac{b-x}{b-a}f(a) + \frac{x-a}{b-a}f(b)$$
$$f(x) \leq \frac{1}{b-a}((b-x)f(a) + (x-a)f(b))$$
$$f(x) - f(a) \leq \frac{1}{b-a}((b-x)f(a) + (x-a)f(b)) \ - f(a)$$
$$x \in (a, b]$$
$$\frac{f(x)-f(a)}{x-a} \leq \frac{1}{(b-a)(x-a)}((b-x)f(a) + (x-a)f(b)) \ - \frac{f(a)}{x-a}$$
$$\cdots \leq \frac{(b-x)}{(b-a)(x-a)}f(a) + \frac{1}{b-a}f(b) - \frac{f(a)}{x-a}$$
$$\cdots \leq \frac{1}{b-a}f(b) + f(a)(\frac{(b-x)}{(b-a)(x-a)} - \frac{(b-a)}{(b-a)(x-a)})$$
$$\cdots \leq \frac{1}{b-a}f(b) + f(a)(\frac{(-1)(x-a)}{(b-a)(x-a)})$$
$$\cdots \leq \frac{1}{b-a}f(b) - \frac{1}{b-a}f(a)$$
$$\frac{f(x)-f(a)}{x-a} \leq \frac{f(b)-f(a)}{b-a} \quad \blacksquare$$

Now for the right side:

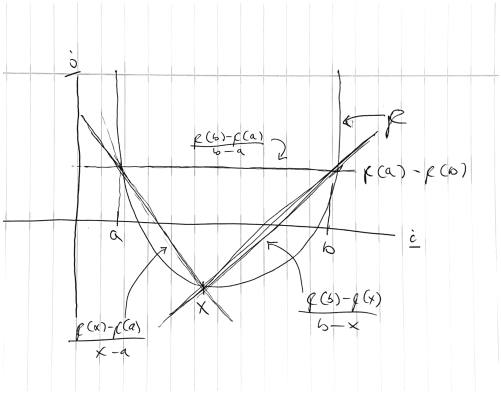
$$f(x) \leq \frac{b-x}{b-a}f(a) + \frac{x-a}{b-a}f(b)$$
$$-f(x) \geq -\frac{b-x}{b-a}f(a) - \frac{x-a}{b-a}f(b)$$
$$f(b) - f(x) \geq f(b) - \frac{b-x}{b-a}f(a) - \frac{x-a}{b-a}f(b)$$
$$x \in [a, b)$$
$$\frac{f(b)-f(x)}{b-x} \geq \frac{f(b)}{b-x} - \frac{b-x}{(b-a)(b-x)}f(a) - \frac{x-a}{(b-a)(b-x)}f(b)$$
$$\cdots \geq \frac{(b-a)}{(b-x)(b-a)}f(b) - \frac{1}{(b-a)}f(a) - \frac{x-a}{(b-a)(b-x)}f(b)$$
$$\cdots \geq f(b)(\frac{b-a-x+a}{(b-x)(b-a)}) - \frac{1}{(b-a)}f(a)$$
$$\cdots \geq f(b)\frac{1}{b-a} - \frac{1}{(b-a)}f(a)$$
$$\frac{f(b)-f(x)}{b-x} \geq \frac{f(b)-f(a)}{b-a} \quad \blacksquare$$

To satisfy both the left and right side we must satisfy $x \in (a, b] \wedge x \in [a, b)$ which is achieved by $x \in (a, b)$. Due too transitivity we may merge the

inequality such that:

$$\frac{f(x)-f(a)}{x-a} \leq \frac{f(b)-f(a)}{b-a} \leq \frac{f(b)-f(x)}{b-x} \quad \blacksquare$$

Which is the inequality I was tasked to show.



(c) Since f is differentiable we know by definition that:

$$\frac{df}{dx}(z) = f'(z) = \lim_{x \to z} \frac{f(z)-f(x)}{z-x}$$

From the previous task we know that for $x \in (a, b)$:

$$\frac{f(x)-f(a)}{x-a} \leq \frac{f(b)-f(a)}{b-a} \leq \frac{f(b)-f(x)}{b-x}$$

Since the inequality is true for the open range (a, b) it must be true for any x arbitrarily close too a and b. The limit order theorem states that if these limits exists (converge) then the inequality also applies to these limits. Therefore we first find the limits:

$$\lim_{x \to a^-} \frac{f(x)-f(a)}{x-a} = f'(a)$$
$$\lim_{x \to b^+} = \frac{f(b)-f(x)}{b-x} = f'(b)$$

Since f is differentiable these limits exists and are equal to the derivative in the boundary. The inequality therefore also applies too the boundary and we have:

$$f'(a) \leq \frac{f(b)-f(a)}{b-a} \leq f'(b) \quad \blacksquare$$

Which is the inequality I was tasked to show.

(d) Since f is twice differentiable we know the derivative of f' exists $f''(x)$. From the previous task we know that for any arbitrary $a, b \in \mathbb{R}$ such that $a > b$ we have:

$$f'(a) \leq \frac{f(b) - f(a)}{b - a} \leq f'(b)$$
$$f'(a) \leq f'(b)$$

Therefore, if we were to construct an arbitrary sequence of real numbers $\{x_n\}_{n\in\mathbb{N}}$ where $x_n \in \mathbb{R}$ and $x_{n+1} > x_n$ we know that the function sequence $\{f'(x_n)\}_{n\in\mathbb{N}}$ is monotonically increasing since $f'(x_n) \leq f'(x_{n+1})$. Since this sequence is monotonically increasing the derivative of the function must always be equal to or larger then 0 (It's always increasing so the derivative, aka. the rate of change must always be positive): $f''(x_n) \geq 0$. Subsisitute x_n for a and b and we have:

$$f''(a) \geq 0, \quad f''(b) \geq 0 \quad \blacksquare$$

Which are the inequalities I was tasked to show.

4. (20 points) (LQR for Buildings) Consider a multi-zone building as in Fig. 2.

(a) Write down the dynamical equations of the building in state-space form, including a term for disturbances $d(t)$, and for the ambient temperature $x_a(t)$, assuming the parameters are

$$\frac{h_{ij}}{C_i} = \begin{cases} 10 \text{ W/°K} & i \text{ is adjacent to } j \\ 0 & \text{else} \end{cases} \tag{7}$$

$$C_i = 1.08 \times 10^6 \text{ J/°K} \tag{8}$$

(b) Consider an LQR cost function of the form

$$\int_0^T \left[(x - \bar{x})^T Q (x - \bar{x}) + u^T R u\right] dt, \tag{9}$$

where $T = 24\text{h} \times 60\frac{\text{m}}{\text{h}} \times 60\frac{\text{s}}{\text{m}} = 86400s$ is one day in seconds. Set $\bar{x} = 20°$, and write the dynamics in the coordinates $z := x - \bar{x}$.

(c) Note that driving the variable $z(t) \to 0$ is therefore equivalent to driving $x(t) \to 20°$. Write a `matlab` script to solve the finite-time horizon LQR problem. Select a positive semi-definite Q and a positive-definite R that gives you "decent" results. Assume the initial conditions, disturbance, and

ambient temperature follow:

$$x_i(0) = 15 - i \times \text{sign}\{(-1)^i\} \quad [^\circ \text{ C}] \tag{10}$$

$$d(t) = \max\left[0, 500 \sin\left(\frac{2\pi t}{86400s}\right) - 300\right] \quad [\text{W}] \tag{11}$$

$$x_a(t) = 20^\circ + 5 \sin\left(\frac{2\pi t}{86400s}\right) \quad [^\circ \text{ C}] \tag{12}$$

where the sign function is:

$$\text{sign}(x) = \begin{cases} 1 & x > 0 \\ -1 & x < 0 \\ 0 & x = 0. \end{cases} \tag{13}$$

Note: If solving the finite-time problem proves too difficult, feel free to solve the infinite-time problem instead. `Matlab` has a built-in function called `lqr` that produces the (constant) gain K; read the documentation for details.

(d) Provide:

- A description of your simulation code
- Plots of $d(t)$, $x_a(t)$, $x_i(t)$, and $u(t)$.
- A discussion of the effects of varying Q and R

Ans:

(a) To derive the state space model of the building's room temperatures, I assume simple dynamics where each room is governed by:

$$\mathring{x}_i = \frac{u_i}{C_i} + \frac{d_i}{C_i} + \sum_{j \in \mathcal{J}_i} \frac{h_{ij}}{C_i}(x_j - x_i)$$

$$[x_i] = K,\ [\mathring{x}_i] = \frac{K}{S},\ [C_i] = \frac{J}{K},\ [\frac{h_{ij}}{C_i}] = \frac{W}{K}$$

$$[u_i] = W,\ [d_i] = [W]$$

$$K = Kelvin,\ S = Secounds,\ J = Joule,\ W = Watt$$

Note that $[\frac{h_{ij}}{C_i}(x_j - x_i)] = [\frac{h_{ij}}{C_i}][(x_j - x_i)] = \frac{W}{K}K = W \neq \frac{K}{S}$ indicating a minor dimensional inconsistency in the provided material. I will assume this is a typographical error and proceed. Also $\mathcal{J}_i \subseteq \{1, 2, 3, 4, 5, a\}$ where $\mathcal{J}_i$ is decided by the neighbors of room i.

Finding the state space model is then just a matter of finding the matrices which evaluates too this equation set.

$$\mathring{x} = Ax + Bu, \quad y = Cx + Du$$

$$x = \begin{pmatrix} x_1 & x_2 & \cdots & x_5 \end{pmatrix}^\top, \quad u = \begin{pmatrix} u' & d & x_a \end{pmatrix}^\top$$

$$u' = (u_i)_{i=1}^3 = \begin{pmatrix} u_1 & u_2 & u_3 \end{pmatrix}, \quad d = (d_i)_{i=1}^5$$

Then the matrices are given by:

$$A = \begin{pmatrix} -\sum_{j\in\mathcal{J}_1}\frac{h_{1,j}}{C_1} & \frac{h_{1,2}}{C_1} & 0 & 0 & \frac{h_{1,5}}{C_1} \\ \frac{h_{2,1}}{C_2} & -\sum_{j\in\mathcal{J}_2}\frac{h_{2,j}}{C_2} & \frac{h_{2,3}}{C_2} & 0 & \frac{h_{2,5}}{C_2} \\ 0 & \frac{h_{3,2}}{C_3} & -\sum_{j\in\mathcal{J}_3}\frac{h_{3,j}}{C_3} & \frac{h_{3,4}}{C_3} & \frac{h_{3,5}}{C_3} \\ 0 & 0 & \frac{h_{4,3}}{C_4} & -\sum_{j\in\mathcal{J}_4}\frac{h_{4,j}}{C_4} & \frac{h_{4,5}}{C_4} \\ \frac{h_{5,1}}{C_5} & \frac{h_{5,2}}{C_5} & \frac{h_{5,3}}{C_5} & \frac{h_{5,4}}{C_5} & -\sum_{j\in\mathcal{J}_5}\frac{h_{5,j}}{C_5} \end{pmatrix}$$

$$B = \begin{pmatrix} \frac{1}{C_1} & 0 & 0 & \frac{1}{C_1} & 0 & 0 & 0 & 0 & \frac{h_{1,a}}{C_1} \\ 0 & \frac{1}{C_2} & 0 & 0 & \frac{1}{C_2} & 0 & 0 & 0 & \frac{h_{2,a}}{C_2} \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{C_3} & 0 & 0 & \frac{h_{3,a}}{C_3} \\ 0 & 0 & \frac{1}{C_4} & 0 & 0 & 0 & \frac{1}{C_4} & 0 & \frac{h_{4,a}}{C_4} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{C_5} & \frac{h_{5,a}}{C_5} \end{pmatrix}$$

$$C = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}, \quad D = 0$$

If we insert all the values we get:

$$A = \begin{pmatrix} -30 & 10 & 0 & 0 & 10 \\ 10 & -40 & 10 & 0 & 10 \\ 0 & 10 & -40 & 10 & 10 \\ 0 & 0 & 10 & -30 & 10 \\ 10 & 10 & 10 & 10 & -50 \end{pmatrix}$$

$$B = \begin{pmatrix} \frac{1}{1.08\cdot 10^6} & 0 & 0 & \frac{1}{1.08\cdot 10^6} & 0 & 0 & 0 & 0 & 10 \\ 0 & \frac{1}{1.08\cdot 10^6} & 0 & 0 & \frac{1}{1.08\cdot 10^6} & 0 & 0 & 0 & 10 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{1.08\cdot 10^6} & 0 & 0 & 10 \\ 0 & 0 & \frac{1}{1.08\cdot 10^6} & 0 & 0 & 0 & \frac{1}{1.08\cdot 10^6} & 0 & 10 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{1.08\cdot 10^6} & 10 \end{pmatrix}$$

I've now provided a state space model of the building's temperature.

(b) In (a) I had to provide the state space model, now I must restate it in $z := x - \bar{x}$. Since $[x] = K$ and $°C + 273 \approx K$ then $\bar{x} = 20°C \approx 293K$ and $z = x - 293$. We notice that $\mathring{z} = x \mathring{-} \bar{x} = \mathring{x}$ and replace:

$$\mathring{x} = Ax + Bu$$
$$\mathring{z} = A(z + \bar{x}) + Bu$$
$$\mathring{z} = Az + Bu + A\bar{x}$$

Which are the dynamics in z. $A\bar{x}$ can we viewed as a constant forcing term. Alternatively you could shift u' or expand the expression to remove the term.

(c) The Code is attached at the very end. My plant is:
Time-invariant: The system matrices A, B are constant, so the system dynamics themselves are time-invariant. However, the disturbance input $d(t)$ is time-varying, which makes the system response time-varying, even though the system structure is not.
Non-linear: For any fixed time t, the disturbance and ambiance acts as a additive constant vector, making the dynamics affine in (x, u'). You can solve the finite time optimization problem for such systems (See: Continuous Nonlinear Optimal Controller with Function of Final State Fixed in Lewis page 115). I'd imagine this would lead to a superior controller since you'd explicitly include a model of the ambiance and disturbance (Internal model principle). But I want to use the LQR functionality in Matlab so I will solve the infinite time problem.

(d) The plots are attached and follow after the solution block. The plant system of the assignment is:

$$\mathring{x} = Ax + Bu, \quad y = Cx$$

These were all defined in part (a). In (b) we restated the system in $z = x - \bar{x}$ and got:

$$\mathring{z} = Az + Bu + A\bar{x}, \quad y_z = Cz$$

Recall that $u = \begin{pmatrix} u' & d & x_a \end{pmatrix}^\top$, we can therefore decompose B into $B =$

$[B_u\ B_d\ B_a]$ where:

$$B_u = \begin{pmatrix} \frac{1}{1.08\cdot 10^6} & 0 & 0 \\ 0 & \frac{1}{1.08\cdot 10^6} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{1.08\cdot 10^6} \\ 0 & 0 & 0 \end{pmatrix}$$

$$B_d = \begin{pmatrix} \frac{1}{1.08\cdot 10^6} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{1.08\cdot 10^6} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{1.08\cdot 10^6} & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{1.08\cdot 10^6} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{1.08\cdot 10^6} \end{pmatrix}$$

$$B_a = \begin{pmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{pmatrix}$$

We can therefore rewrite our system in z too:

$$\mathring{z} = Az + B_u u' + B_d d + B_a x_a + A\bar{x}$$

$$\mathring{z} = Az + B_u u' + B_d d + \begin{pmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{pmatrix} (293 + 5\sin\left(\frac{2\pi t}{86400s}\right)) + \begin{pmatrix} -10 \\ -10 \\ -10 \\ -10 \\ -10 \end{pmatrix} (293)$$

$$\mathring{z} = Az + B_u u' + B_d d + \begin{pmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{pmatrix} (5\sin\left(\frac{2\pi t}{86400s}\right))$$

This will be the system model I'll work on. The first part of my code is just to define these and the initial condition. Since the initial condition provided was $x(0) = \begin{pmatrix} 289 & 286 & 291 & 284 & 293 \end{pmatrix}^\top$ in Kelvin, then $z(0) = x(0) - \bar{x} = \begin{pmatrix} -4 & -7 & -2 & -9 & 0 \end{pmatrix}^\top$.

To solve the free-final state LQR problem using Matlab's functionality, I need an LTI system. I therefore find the steady state feedback gain K of my feedback controller $u' = -Kz$ by solving using the system:

$$\mathring{z} = Az + B_u u'$$

Which is our system in z without any external input (disturbance and ambient temperature). By removing the ambient temperature and not adjusting the A matrix accordingly I have effectively set the ambient temperature too $20°C$ in this model.

After I've calculated K I want to simulate the entire system with this controller, $u' = -K\hat{z}$. Since my plant output is not the entire state, I must design an observer. Luckily the system is observable and my observer is: $\mathring{\hat{z}} = A\hat{z} - B_u K\hat{z} + B_d d + B_a x_a + L(y - C\hat{z}),\ \hat{z}(0) = 0$. This observer is created using the eigenvalue assignment (pole placement) method, which gives a gain L. My system is now:

$$\frac{\partial}{\partial t}\begin{pmatrix} z \\ \hat{z} \end{pmatrix} = \begin{pmatrix} A & -B_u K \\ LC & A - B_u K - LC \end{pmatrix}\begin{pmatrix} z \\ \hat{z} \end{pmatrix} + \begin{pmatrix} B_d & B_a \\ B_d & B_a \end{pmatrix}\begin{pmatrix} d \\ x_a \end{pmatrix}$$

Which I simulate for the first 24 hours, $t \in [0, 86400] \cap \mathbb{N}$.

Since the gain K is determined by minimizing J which is functionally dependent on Q and R my controller behaves differently depending on the Q and R I choose. Assume I is the identity matrix and s, r are scalars while $Q = Is$ and $R = Ir$. By increasing the spread $(s - r)$ I am penalizing state deviations from 0 more then the control deviations from 0. This leads to a more aggressive or responsive controller which outputs large control vectors to force the system to 0. If i decrease the spread the controller won't fight state drifts away from 0 as fiercely, and will in our case when $s \approx r$ allow the room temperatures to track the ambient temperature. Generally Q and R mustn't be diagonal but the same principle applies generally only you can weigh how deviations in individual vector elements or cross terms are weighed relative to each other.

Therefore generally by "increasing" Q you prioritize keeping the state around 0. By "increasing" R you prioritize keeping the control around 0.

Btw the control being negative implies negative watts which doesn't make sense but if you interpret positive as heating and negative as cooling it does.

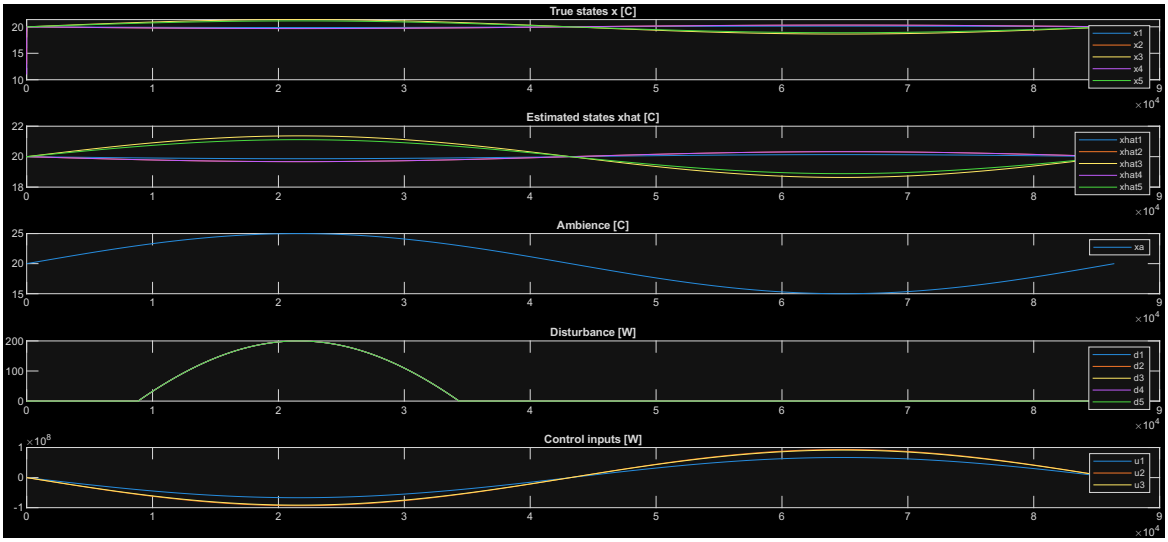


Figure 1: Plots of d(t), xa(t), xi(t), and u(t). First 24 hours.

5. (10 points) Suppose that $Q = \mathbf{0}$, the zero matrix, and $R = I$. Prove that

$$x_0^T S_0 x_0 = \min_{\{u_k\}_{k=0}^{T-1}} \sum_{k=0}^{T-1} \|u_k\|_2^2, \tag{14}$$

meaning that $J_0 = x_0^T S_0 x_0$ is the minimum control energy required to drive the system to zero.

Ans: Assume we're given a discrete in time, LTI system and a cost index:

$$x_{k+1} = Ax_k + Bu_k$$
$$J_i = x_N S_N x_N + \sum_{k=i}^{N-1} x_k^\top Q x_k + u_k^\top R u_k$$

We want to find the control sequence $u^* = (u_k^*)_{k=0}^{N-1}$ which minimizes J_0 such that the dynamics and initial condition are respected. This minimum is J_0^* and the problem is formulated as:

$$J_0^* = \min_u \; J_0(u)$$
$$s.t. \quad x_{k+1} = Ax_k + Bu_k$$
$$x_0 = x_{IC}$$

I can prove the solution to this problem, but since it essentially is a copy of page 270 – 271 in Optimal Control by Lewis, I'll merely state it (My proof

would've been the exact same, just without the multiplication by: $\frac{1}{2}$). Assume that $S_N, Q \geq 0, R > 0$, then we can using Bellman's principle of optimality and the convexity of J_i with respect to u show that the solution is given by:

$$u^*_{n-1} = -K_{n-1}x_{n-1}$$
$$S_{n-1} = (A - BK_{n-1})^\top S_n(A - BK_{n-1}) + K^\top_{n-1}RK_{n-1} + Q$$
$$K_{n-1} = (2R + 2B^\top S_n B)^{-1}2B^\top S_n A$$
$$J^*_0 = x_0^\top S_0 x_0$$

Let's now find the optimal cost index J^*_0 for a specific scenario where:

$$x_{k+1} = Ax_k + Bu_k$$
$$J_i = x_N S_N x_N + \sum_{k=i}^{N-1} x_k^\top Q x_k + u_k^\top R u_k$$
$$x_N = 0,\ Q = 0,\ R = I$$

Then we know that:

$$J^*_0 = x_0^\top S_0 x_0 = x_N S_N x_N + \sum_{k=0}^{N-1} x_k^\top Q x_k + (u^*_k)^\top R u^*_k$$
$$= \sum_{k=0}^{N-1} (u^*_k)^\top u^*_k = \sum_{k=0}^{N-1} ||u^*_k||_2^2$$

If we additionally use the fact that: $J^*_0 = \min_u J_0(u)$ we get:

$$x_0^\top S_0 x_0 = \sum_{k=0}^{N-1} ||u^*_k||_2^2 = \min_u \sum_{k=0}^{N-1} ||u_k||_2^2 \quad \blacksquare$$

Which is the equality I was tasked to prove. Note with regards to notation: $u = (u_k)_{k=0}^{N-1}$ and I define this such that $(u_k)_{k=0}^{N-1} = \{u_k\}_{k=0}^{N-1}$. I just prefer the parenthesis since you emphasize that the order of elements matter. Some may confuse the curly braces with a set, in which order becomes irrelevant.

6. (10 points) *Crassidis & Junkins* Problem 3.14

Ans: The Code is attached at the very end. The plots follow after the solution block. I've solved the problem for the scenarios (1) (2) and (3)

- (1) Initial state expectation $[0, 0]$ and covariance $\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$
- (2) Initial state expectation $[1, 1]$ and covariance $\begin{pmatrix} 0.01 & 0.01 \\ 0.01 & 0.01 \end{pmatrix}$
- (3) Initial state expectation $[100, 100]$ and covariance $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$

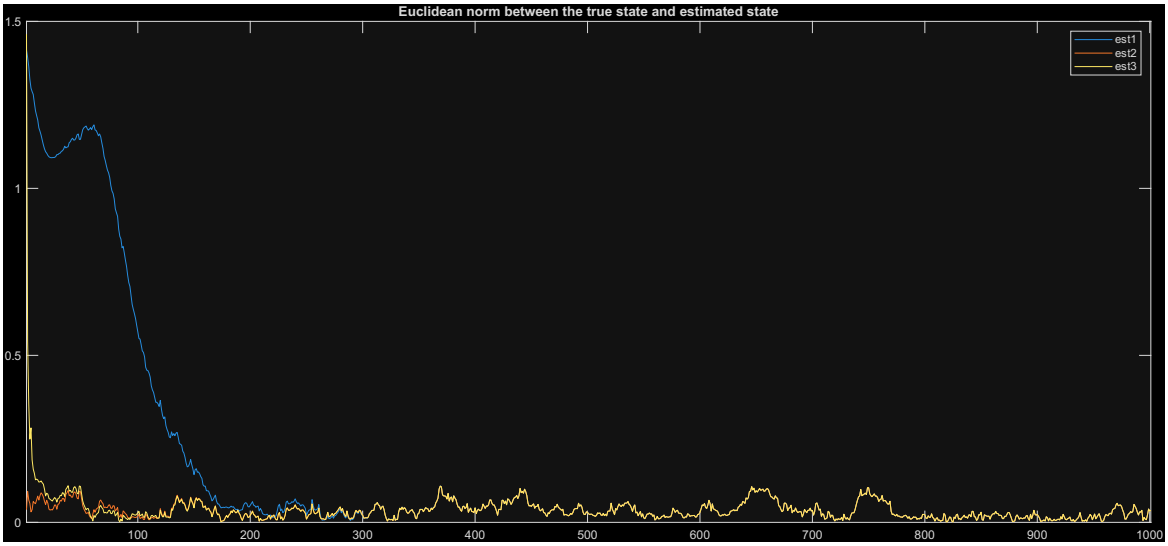


Figure 2: The distance between the estimated state and the true state for each scenario

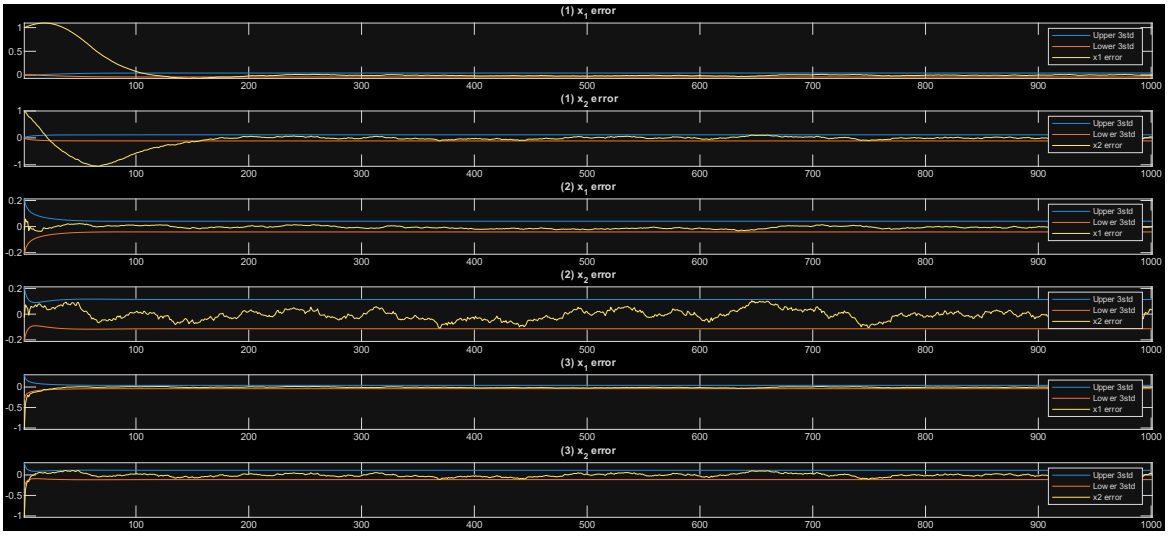


Figure 3: The state estimate error and 3 sigma bounds for each state and scenario

7. (10 points) *Crassidis & Junkins* Problem 3.15

Ans: The Code is attached at the very end. The plots follow after the solution block. The problem has been solved for the same scenarios as in the previous question. I'll assess whether the change in measurement model yielded improved state estimates visually through the plots. The steady state estimation error is improved in the second model. But in scenario (1) it takes approximately twice as much time too reach the steady state, degrading estimation. Though scenario (1) is a heinous case where we've incorrectly stated the initial state is 0 with absolute certainty. We can also see that the 3 sigma bounds are narrower in the second model. Overall the state estimates appear to be improved, which you'd expect due to the increased amount of information informing the estimate.

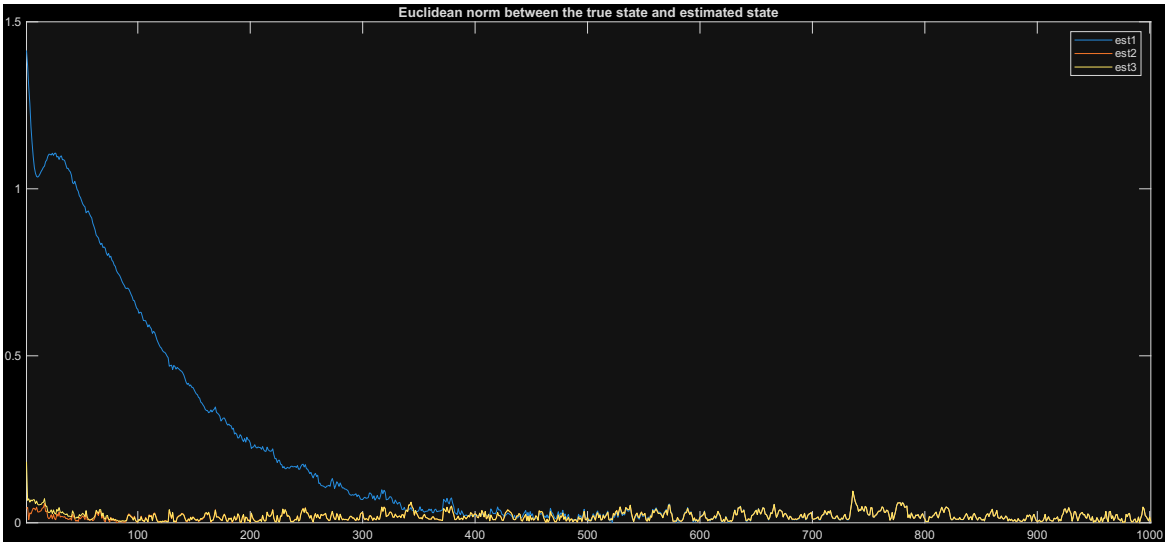


Figure 4: The distance between the estimated state and the true state for each scenario

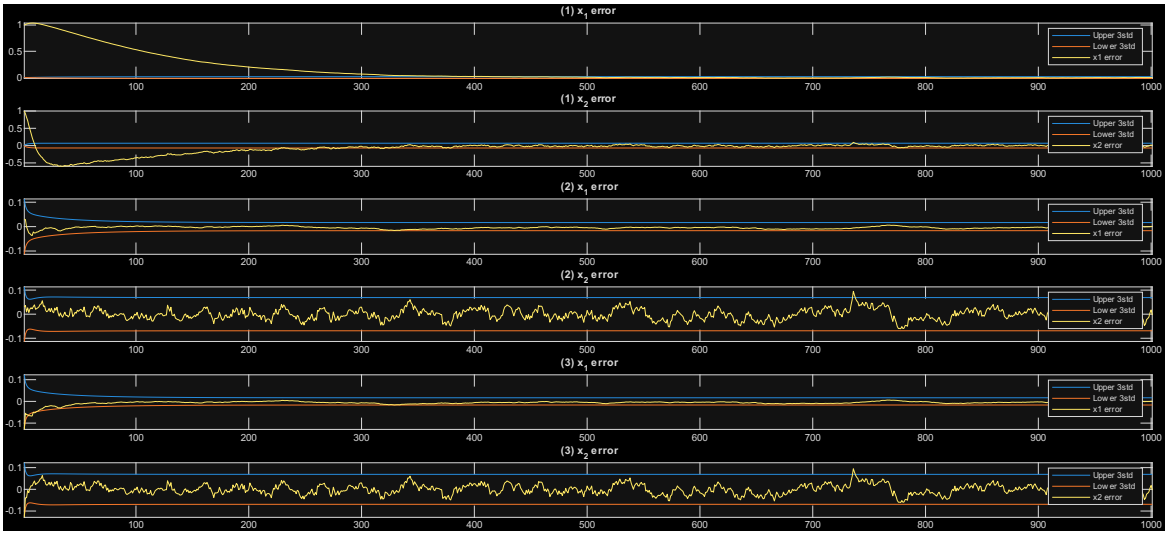


Figure 5: The state estimate error and 3 sigma bounds for each state and scenario

8. (10 points) *Crassidis & Junkins* Problem 7.4

Ans: The Code is attached at the very end. The plots follow after the solution block.

The assignment is to simulate and estimate the attitude of a spacecraft in orbit using the extended Kalman filter for attitude estimation. The spacecraft's attitude (represented as a quaternion q) changes over time due to an angular velocity ω. To estimate the attitude we're provided with an attitude measurement (derived from a star camera) and an angular velocity estimate (derived from a gyro). The system and conditions must reflect example 7.1 and 6.1 in Crassidis. I must also run the simulation with varying process noise in the gyro bias and measurement noise in the gyro measurement, then assess performance in each scenario, then assess which source of noise affects performance the most.

The system to be simulated is:

$$\mathring{x} = \begin{pmatrix} \mathring{q} \\ \mathring{\beta} \end{pmatrix} = \begin{pmatrix} \frac{1}{2}\Omega(\omega) & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} q \\ \beta \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \eta_u$$

$$\eta_u \sim \mathcal{N}(0, \sigma_u^2 I_3)$$

$$x(0) = \begin{pmatrix} q_0 \\ \beta_0 \end{pmatrix}, \; q_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}, \; \beta_0 = \begin{pmatrix} 0.1 \\ 0.1 \\ 0.1 \end{pmatrix}$$

With the definitions:

$$q = \begin{pmatrix} \varrho \\ q_4 \end{pmatrix} \in \mathbb{R}^4 \; s.t. \; q^\top q = 1$$

$$\frac{1}{2}\Omega(\omega)q = \frac{1}{2}\begin{pmatrix} \omega \\ 0 \end{pmatrix} \otimes q, \quad q' \otimes q = \begin{pmatrix} \Psi(q') & q' \end{pmatrix} q$$

$$\Psi(q) = \begin{pmatrix} q_4 I_3 - [\varrho\times] \\ -\varrho^\top \end{pmatrix}$$

$$\mathbf{a} = (a_1, a_2, a_3)^\top \in \mathbb{R}^3, \quad [\mathbf{a}\times] = \begin{pmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{pmatrix}$$

The reference frame $(\mathbf{r}_i)_{i=1}^3$ is the standard basis $\mathbf{r}_i = e_i$ and the body frame is $(\mathbf{b}_i)_{i=1}^3$ which are related by $\mathbf{b}_i = A(q)\mathbf{r}_i$. The initial attitude q_0 is such that the body frame is aligned with the reference frame $\mathbf{b}_i = \mathbf{r}_i$, and the initial gyro bias β_0 is 0. The angular velocity is assumed constant such that there is only rotation about the $\mathbf{r}_3$ reference basis vector. The speed of rotation is such that a whole rotation is completed after 90 minutes, since the orbit is a 90-minute orbit and our star camera points out toward the stars, hence $\omega = \begin{pmatrix} 0 & 0 & \frac{\pi}{2700} \end{pmatrix}^\top$.

The true system is continuous in time, but the measurement model is discrete in time. Measurements are sampled each second. The measurement model is:

$$\tilde{y}[k] = \begin{pmatrix} \tilde{\mathbf{b}}[k] \\ \tilde{\omega}[k] \end{pmatrix}$$

$$\tilde{\mathbf{b}}[k] = \begin{pmatrix} A(q(t_k))\mathbf{r}_1 \\ A(q(t_k))\mathbf{r}_2 \\ A(q(t_k))\mathbf{r}_3 \end{pmatrix} + \begin{pmatrix} z_1(t_k) \\ z_2(t_k) \\ z_3(t_k) \end{pmatrix}, \quad z_i \sim \mathcal{N}(0, \sigma_i^2 I_3)$$

Re-normalize after adding noise

$$\tilde{\omega}[k] = \omega + \beta(t_k) + \eta_v, \quad \eta_v \sim \mathcal{N}(0, \sigma_v^2 I_3)$$

With the definitions:

$$A(q) = \Xi(q)^\top \Psi(q), \quad \Xi(q) = \begin{pmatrix} q_4 I_3 + [\varrho\times] \\ -\varrho^\top \end{pmatrix}$$

$$R = \begin{pmatrix} \sigma_1^2 I_3 & 0 & 0 \\ 0 & \sigma_2^2 I_3 & 0 \\ 0 & 0 & \sigma_3^2 I_3 \end{pmatrix}$$

We'll simulate this system for the first 90 minutes. To simulate the system we must first note that the quaternion must satisfy $q^\top q = 1$ or at-least $\sqrt{q^\top q}$ –

$1 \ll \delta$ where $\delta \in [0, 10^{-8}]$. The upper bound is chosen such that error is negligible and I've chosen 10^{-8} as to not markedly increase complexity which, a proper handling of quaternion integration (see Lie group integration) would entail.

To satisfy this unit length requirement I'll subdivide the total simulation period $[0, T]$ into sub-periods by choosing an $M \in \mathbb{N}$ such that:

$$[0, T] = \bigcup_{n=0}^{N-1} [t_n, t_{n+1}], \quad t_n = \frac{1}{M} n$$
$$N = T \cdot M$$

Every whole second becomes an endpoint in a sub-period which is needed for the measurement model. At the start of each sub-period we draw the random variable η_u and normalize the quaternion $\frac{q}{||q||}$ then we simulate the system over the interval using Matlab's ODE solvers. The solver returns $x(t_{n+1})$ which becomes the start of the next interval and we repeat. We therefore stop up every once in a while to re-normalize the quaternion and hold the stochastic component fixed over the same intervals. This returns the trajectory of x on $[0, T]$ which we then feed into our measurement model to get the sequence $\tilde{y}$.

Now that we have the measurement sequence, we must define our observer, the extended Kalman filter for attitude estimation. Our observer is initialized with the expectations $\hat{q}(0) = \hat{q}_0$ and $\hat{\beta}(0) = \hat{\beta}_0$ and the error covariance $P(0) = P_0$ which is 6×6 matrix.

Gain

$$K_k = P_k^- H_k^\top (H_k P_k^- H_k^\top + R)^{-1}$$

Update

$$\Delta\hat{\tilde{x}}_k^+ = K_k(\tilde{\mathbf{b}}_k - h_k) = \begin{pmatrix} \delta\hat{\alpha}_k^+ \\ \Delta\hat{\beta}_k^+ \end{pmatrix}$$
$$P_k^+ = (I_6 - K_k H_k) P_k^-$$
$$\hat{q}_k^+ = \hat{q}_k^- + \frac{1}{2}\Xi(\hat{q}_k^-)\delta\hat{\alpha}_k^+, \quad \text{Re-Normalize}$$
$$\hat{\beta}_k^+ = \hat{\beta}_k^- + \Delta\hat{\beta}_k^+$$
$$\hat{\omega}_k^+ = \tilde{\omega}_k - \hat{\beta}_k^+$$

Propogate

$$\hat{q}_{k+1}^- = \bar{\Omega}(\hat{\omega}_k^+)\hat{q}_k^+, \quad \hat{\beta}_{k+1}^- = \hat{\beta}_k^+$$
$$P_{k+1}^- = \Phi_k P_k^+ \Phi_k^\top + \Upsilon_k Q_k \Upsilon_k^\top$$

With the definitions:

$$H_k = \begin{pmatrix} [A(\hat{q}_k^-)r_1\times] & 0_3 \\ [A(\hat{q}_k^-)r_2\times] & 0_3 \\ [A(\hat{q}_k^-)r_3\times] & 0_3 \end{pmatrix}, \quad h_k = \begin{pmatrix} A(\hat{q}_k^-)r_1 \\ A(\hat{q}_k^-)r_2 \\ A(\hat{q}_k^-)r_3 \end{pmatrix}, \quad \alpha = (\angle roll, \angle pitch, \angle yaw)^\top$$

$$\bar{\Omega}(\hat{\omega}_k^+) = \begin{pmatrix} cos(\frac{1}{2}||\hat{\omega}_k^+||\Delta t)I_3 - [\hat{\psi}_k^+\times] & \hat{\psi}_k^+ \\ -(\hat{\psi}_k^+)^\top & cos(\frac{1}{2}||\hat{\omega}_k^+||\Delta t) \end{pmatrix}$$

$$\hat{\psi}_k^+ = \frac{sin(\frac{1}{2}||\hat{\omega}_k^+||\Delta t)\hat{\omega}_k^+}{||\hat{\omega}_k^+||}$$

$$\Upsilon_k = \begin{pmatrix} -I_3 & 0_3 \\ 0_3 & I_3 \end{pmatrix}, \quad \Delta t = \text{Gyro sampling interval}$$

$$\Phi_k = \begin{pmatrix} \Phi_k^{11} & \Phi_k^{12} \\ \Phi_k^{21} & \Phi_k^{22} \end{pmatrix}$$

$$\Phi_k^{11} = I_3 - [\hat{\omega}_k^+\times]\frac{sin(||\hat{\omega}_k^+||\Delta t)}{||\hat{\omega}_k^+||} + [\hat{\omega}_k^+\times]^2\frac{(1 - cos(||\hat{\omega}_k^+||\Delta t))}{||\hat{\omega}_k^+||^2}$$

$$\Phi_k^{12} = [\hat{\omega}_k^+\times]\frac{(1 - cos(||\hat{\omega}_k^+||\Delta t))}{||\hat{\omega}_k^+||^2} - I_3\Delta t - [\hat{\omega}_k^+\times]^2\frac{(||\omega_k^+||\Delta t - sin(||\hat{\omega}_k^+||\Delta t))}{||\hat{\omega}_k^+||^3}$$

$$\Phi_k^{21} = 0_3, \quad \Phi_k^{22} = I_3$$

$$Q_k = \begin{pmatrix} (\sigma_v^2\Delta t + \frac{1}{3}\sigma_u^2\Delta t^3)I_3 & (\frac{1}{2}\sigma_u^2\Delta t^2)I_3 \\ (\frac{1}{2}\sigma_u^2\Delta t^2)I_3 & (\sigma_u^2\Delta t)I_3 \end{pmatrix}$$

Now that I've implemented our filter and produced our estimates I must evaluate the performance of the filter for various σ_v, σ_u, which ones seem to have the largest effect on performance?

The plots follow after the solution block. The scenarios are as follows:

- Scenario 1: $\sigma_u = 1, \sigma_v = 1$
- Scenario 2: $\sigma_u = 1, \sigma_v = 2$
- Scenario 3: $\sigma_u = 2, \sigma_v = 1$

Besides the differences in gyro variances, every other parameter is equal. σ_u was related to the gyro bias drift and σ_v to the gyro measurement noise. From the plots we can see that the attitude is well estimated across all 3 scenarios. The Forbenius norm between the attitude matrices were generally

below 3 (though very jittery, likely due to the error being aggregated across roll, pitch and jaw) and the average variance of Euler angle residuals are low, indicating high certainty in measurements (small residuals). The gyro bias estimate and angular velocity varies more between the scenarios. With regards to gyro bias, scenario 1 does the best, as it has the least noise, while scenario 3 performs the worst. This indicates that bias drift impacts bias estimates more then measurement noise. But scenario 3 has lower Euler angle residuals then scenario 2, indicating that measurement noise impacts attitude estimation more then bias drift.

To summarize σ_v, σ_u impacts performance differently. While σ_u appears to impact the bias estimate to a larger extent, σ_v impacts the attitude estimate to a larger extent.

Figure 6: The distance between the estimated state and the true state for each scenario

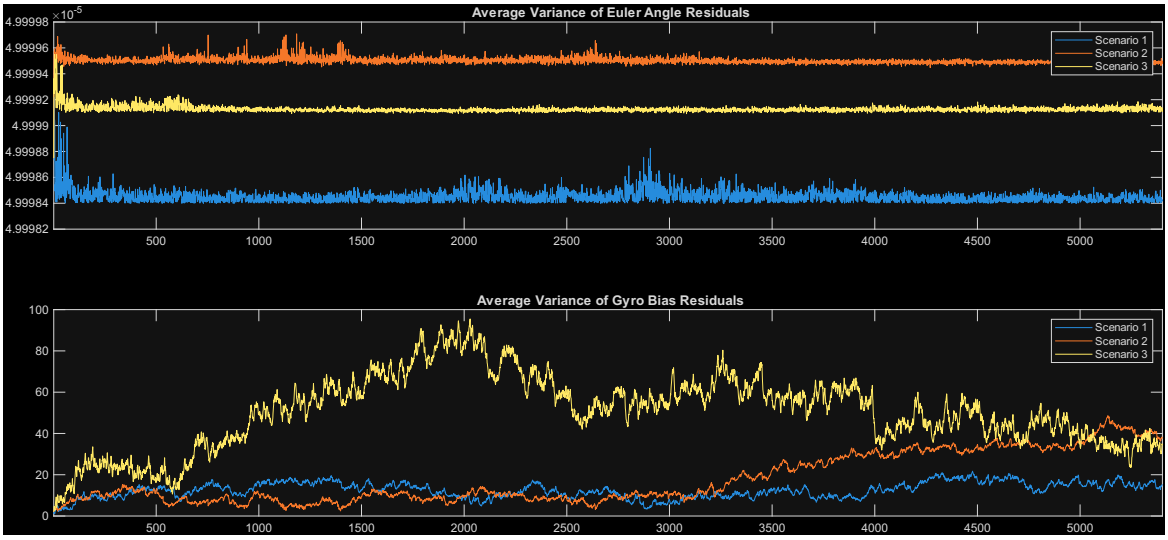


Figure 7: The average variances for each state and scenario

Code

```
%% Q4

%Define model and parameters
%C_i inverse
c = 1/(1.08*10^6); %[K/J]

% \bar{x}
x_bar = [0;0;0;0;0] + 20 + 273; %[K]

%Initial condition
x_ic = [16;13;18;11;20] + 273; %[K]

%z = x-\bar{x}
z_ic = x_ic - x_bar; %[K]

%Disturbance: d(t)
function d_t = dtr(t)
    d = 500*sin(2*pi*t/86400)-300;
    d_t = max(0,d); %[W]
end

%Ambient Temperature: x_a(t) - 20
function xa_t = amb(t)
    xa_t = 5*sin(2*pi*t/86400);
end

%System matrix: A
A = [-30 10 0 0 10;
    10 -40 10 0 10;
    0 10 -40 10 10;
    0 0 10 -30 10;
    10 10 10 10 -50];

%Controllable forcing matrix: B_u
B_u = [c 0 0; 0 c 0; 0 0 0; 0 0 c; 0 0 0];
%Disturbance matrix: B_d
B_d = eye(5)*c;
%Ambience matrix: B_{x_a}
B_a = [10; 10; 10; 10; 10];
%Forcing matrix: B
B = [B_u B_d B_a];
%Output matrix: C
C = [1 0 0 0 0; 0 0 1 0 0];
```

```
%Pass-through matrix: D
D=zeros(2,3);

%Find the state feedbacklaw on z (K). Solve the free final
    state LQR (on z)
Q = eye(5)*1e10;
R = eye(3)*1e-10;
lqr_sys = ss(A,B_u,C,D);
[K,S,P] = lqr(lqr_sys,Q,R);

% Observer
% check observability first
if rank(obsv(A, C)) == size(A,1)
    disp('Observable')
else
    error('Not observable')
end
% Place poles
desired_obs_poles = [ -10, -12, -15, -20, -25 ];   % must
   be faster than closed-loop
L = place(A', C', desired_obs_poles)';

% Simulate the system on z.
% Augmented system for state vector [z; zhat]
A_aug = [ A,           -B_u*K;
          L*C,  A - B_u*K - L*C ];

B_ext_top = [B_d, B_a];
B_ext_bottom = [B_d, B_a];
B_aug = [ B_ext_top;
          B_ext_bottom ];

C_aug = [ C, zeros(size(C)) ];
D_aug = zeros(size(C_aug,1), size(B_aug,2));

sys_aug = ss(A_aug, B_aug, C_aug, D_aug);

% initial conditions
zhat_ic = zeros(size(z_ic));     % choose as desired
z_aug_ic = [ z_ic; zhat_ic ];

% time vector (1-second steps for the first 24 hours)
t = 0:1:86400;

% build external input u_ext(t) = [d1..d5, xa] for every t
```

```
87  % vectorized: compute d(t) and xa(t) using the provided
       functions
88  d_vec = arrayfun(@dtr, t(:));
89  xa_vec = arrayfun(@amb, t(:));
90  % form the input matrix (N x 6)
91  u_ext = [ repmat(d_vec, 1, 5), xa_vec ];  % columns: d1 d2
        d3 d4 d5 xa
92
93  % simulate
94  [y_out, t_out, z_aug_traj] = lsim(sys_aug, u_ext, t,
       z_aug_ic);
95
96  % extract trajectories
97  n = size(A,1);
98  z_traj = z_aug_traj(:, 1:n);           % true states (N x
       5)
99  zhat_traj = z_aug_traj(:, n+1:end);    % estimated states
       (N x 5)
100 y_traj = y_out;                        % plant outputs (N
       x 5)
101
102 % compute internal control signal u_ctrl(t) = -K * zhat(t)
103 % K is 3x5, zhat_traj is N x 5 -> u_traj (N x 3)
104 u_ctrl_traj = -( zhat_traj * K' );
105
106 %Plots of d(t), xa(t), xi(t), and u(t).
107 figure;
108 subplot(5,1,1); plot(t_out, z_traj + 20); title('True
       states x [C]'); legend('x1','x2','x3','x4','x5');
109 subplot(5,1,2); plot(t_out, zhat_traj + 20); title('
       Estimated states xhat [C]'); legend('xhat1','xhat2','
       xhat3','xhat4','xhat5');
110 subplot(5,1,3); plot(t_out, xa_vec + 20); title('Ambience
       [C]'); legend( xa );
111 subplot(5,1,4); plot(t_out, repmat(d_vec, 1, 5)); title('
       Disturbance [W]'); legend('d1','d2','d3','d4','d5');
112 subplot(5,1,5); plot(t_out, u_ctrl_traj); title('Control
       inputs [W]'); legend('u1','u2','u3');
113
114
115 %% Q6
116 rng('default') % For reproducibility
117
118 %Implement model
119 A = [9.9985*10^-1 9.8510*10^-3;
```

```
    -2.9553*10^-2 9.7030*10^-1];
B = [4.9502*10^-5;
    9.8510*10^-3];
C = [1 0];
Q = 1;
R = 0.01;

function out = w_k(Q)
    out = randn(1)*sqrt(Q);
end

function out = v_k(R)
    out = randn(1)*sqrt(R);
end


%Simulate model for k in {0,1,2,...,1000}
N = 1000;
N = N + 1;

%Initial conditions
x = zeros(2, N);
y = zeros(1, N);
y_n = zeros(1, N);
x(:, 1) = [1; 1];
y(1) = 1;
y_n(1) = 1 + v_k(R);

for k = 1:N-1
    x_k = x(:, k);
    x_k1 = A*x_k + B*w_k(Q);
    x(:, k+1) = x_k1;
    y(k+1) = C*x_k1;
    y_n(k+1) =  y(k+1) + v_k(R);
end

%Estimate the state using a Kalman filter with varying
   initial values.
function [est, err] = kalfil(y_n,A,B,C,Q,R, xo, Po)
    %{
    Args:
        y_n - A vector of system outputs (1xN)
        A,B,C - System matrices
        Q - Process noise covariance (scalar)
        R - Measurement noise covariance (scalar)
```

```
164         xo - Initial state expectation (2x1)
165         Po - Initial error covariance (2x2)
166     Returns:
167         est: A sequence of state estimates (Only updated)
              (2xN)
168         err: A sequence of error covariances (2x2xN)
169     %}
170     N = length(y_n);
171     est = zeros(2,N);
172     err = zeros(2,2,N);
173
174     %Initialize for k=0
175     %Gain
176     K_k = Po*(C.')*inv(C*Po*C.' + R);
177     %Update
178     x_k_p = xo + K_k*(y_n(1) - C*xo);
179     P_k_p = (eye(2)-K_k*C)*Po;
180     %Propogate
181     x_k1_m = A*x_k_p;
182     P_k1_m = A*P_k_p*A.' + B*Q*B.';
183     %Save
184     est(:,1) = x_k_p;
185     err(:,:,1) = P_k_p;
186     %Run for k=1 too k=N-1
187     for k = 2:N-1
188         %Get
189         P_k_m = P_k1_m;
190         x_k_m = x_k1_m;
191         %Gain
192         K_k = P_k_m*(C.')*inv(C*P_k_m*C.' + R);
193         %Update
194         x_k_p = x_k_m + K_k*(y_n(k) - C*x_k_m);
195         P_k_p = (eye(2)-K_k*C)*P_k_m;
196         %Propogate
197         x_k1_m = A*x_k_p;
198         P_k1_m = A*P_k_p*A.' + B*Q*B.';
199         %Save
200         est(:,k) = x_k_p;
201         err(:,:,k) = P_k_p;
202     end
203     %End for k=N
204     %Get
205     P_k_m = P_k1_m;
206     x_k_m = x_k1_m;
207     %Gain
```

```
208     K_k = P_k_m*(C.')*inv(C*P_k_m*C.' + R);
209     %Update
210     x_k_p = x_k_m + K_k*(y_n(k) - C*x_k_m);
211     P_k_p = (eye(2)-K_k*C)*P_k_m;
212     %Save
213     est(:,N) = x_k_p;
214     err(:,:,N) = P_k_p;
215     %Return
216 end
217
218 [est_1,err_1] = kalfil(y_n,A,B,C,Q,R, [0,0].', [0 0; 0 0])
        ;
219 [est_2,err_2] = kalfil(y_n,A,B,C,Q,R, [1,1].', [0.01 0.01;
        0.01 0.01]);
220 [est_3,err_3] = kalfil(y_n,A,B,C,Q,R, [100,100].', [1 1; 1
        1]);
221
222 %Test covergance (estimate vs true state) with each
        varying initial value
223 %Calculate the Euclidean norm between the true state and
        estimated state.
224 %Calculate the state errors
225 diff1 = x - est_1;
226 diff2 = x - est_2;
227 diff3 = x - est_3;
228 dist1 = sqrt(sum((diff1).^2));
229 dist2 = sqrt(sum((diff2).^2));
230 dist3 = sqrt(sum((diff3).^2));
231 %plot norms
232 figure;
233 plot(1:N,[dist1.' dist2.' dist3.'] ); title('Euclidean
        norm between the true state and estimated state');
        legend('est1','est2','est3');
234 xlim([1, N]);
235
236 %Compare the state errors with their respective 3thetha
        bounds computed from the covariance matrix Pk
237 %Compute the 3thetha bounds
238 x1_3std_err_1 = sqrt(squeeze(err_1(1,1,:)))*3;
239 x2_3std_err_1 = sqrt(squeeze(err_1(2,2,:)))*3;
240
241 x1_3std_err_2 = sqrt(squeeze(err_2(1,1,:)))*3;
242 x2_3std_err_2 = sqrt(squeeze(err_2(2,2,:)))*3;
243
244 x1_3std_err_3 = sqrt(squeeze(err_3(1,1,:)))*3;
```

```
245 x2_3std_err_3 = sqrt(squeeze(err_3(2,2,:)))*3;
246
247 %Extract the errors per state
248 x1_err_1 = diff1(1,:);
249 x2_err_1 = diff1(2,:);
250
251 x1_err_2 = diff2(1,:);
252 x2_err_2 = diff2(2,:);
253
254 x1_err_3 = diff3(1,:);
255 x2_err_3 = diff3(2,:);
256
257 %Plot
258 figure;
259 subplot(6,1,1); plot(1:N, [x1_3std_err_1, (-1)*
       x1_3std_err_1, x1_err_1.']); title('(1) x_1 error');
       legend('Upper 3std','Lower 3std','x1 error'); xlim([1, N
       ]);
260 subplot(6,1,2); plot(1:N, [x2_3std_err_1, (-1)*
       x2_3std_err_1, x2_err_1.']); title('(1) x_2 error');
       legend('Upper 3std','Lower 3std','x2 error'); xlim([1, N
       ]);
261 subplot(6,1,3); plot(1:N, [x1_3std_err_2, (-1)*
       x1_3std_err_2, x1_err_2.']); title('(2) x_1 error');
       legend('Upper 3std','Lower 3std','x1 error'); xlim([1, N
       ]);
262 subplot(6,1,4); plot(1:N, [x2_3std_err_2, (-1)*
       x2_3std_err_2, x2_err_2.']); title('(2) x_2 error');
       legend('Upper 3std','Lower 3std','x2 error'); xlim([1, N
       ]);
263 subplot(6,1,5); plot(1:N, [x1_3std_err_3, (-1)*
       x1_3std_err_3, x1_err_3.']); title('(3) x_1 error');
       legend('Upper 3std','Lower 3std','x1 error'); xlim([1, N
       ]);
264 subplot(6,1,6); plot(1:N, [x2_3std_err_3, (-1)*
       x2_3std_err_3, x2_err_3.']); title('(3) x_2 error');
       legend('Upper 3std','Lower 3std','x2 error'); xlim([1, N
       ]);
265
266 %% Q7
267
268 rng('default') % For reproducibility
269
270 %Implement model
271 A = [9.9985*10^-1 9.8510*10^-3;
```

```
    -2.9553*10^-2 9.7030*10^-1];
B = [4.9502*10^-5;
    9.8510*10^-3];
C = [1 0;0 1; 1 1];
Q = 1;
R = eye(3)*0.01;

function out = v_k2(R)
    v1 = randn(1)*sqrt(0.01);
    v2 = randn(1)*sqrt(0.01);
    v3 = randn(1)*sqrt(0.01);
    out = [v1; v2; v3];
end


%Simulate model for k in {0,1,2,...,1000}
N = 1000;
N = N + 1;

%Initial conditions
x = zeros(2, N);
y = zeros(3, N);
y_n = zeros(3, N);
x(:, 1) = [1; 1];
y(:,1) = C*[1; 1];
y_n(:,1) = C*[1; 1] + v_k2(R);

for k = 1:N-1
    x_k = x(:, k);
    x_k1 = A*x_k + B*w_k(Q);
    x(:, k+1) = x_k1;
    y(:,k+1) = C*x_k1;
    y_n(:,k+1) =  y(:,k+1) + v_k2(R);
end

%Estimate the state using a Kalman filter with varying
   initial values.
function [est, err] = kalfil2(y_n,A,B,C,Q,R, xo, Po)
    %{
    Args:
        y_n - A vector of system outputs (3xN)
        A,B,C - System matrices
        Q - Process noise covariance (scalar)
        R - Measurement noise covariance (3x3) (Diagonal
           matrix)
```

```
315         xo - Initial state expectation (2x1)
316         Po - Initial error covariance (2x2)
317     Returns:
318         est: A sequence of state estimates (Only updated)
               (2xN)
319         err: A sequence of error covariances (2x2xN)
320     %}
321     N = length(y_n);
322     est = zeros(2,N);
323     err = zeros(2,2,N);
324
325     %Initialize for k=0
326     %Gain
327     K_k = Po*(C.')*inv(C*Po*C.' + R);
328     %Update
329     x_k_p = xo + K_k*(y_n(:,1) - C*xo);
330     P_k_p = (eye(2)-K_k*C)*Po;
331     %Propogate
332     x_k1_m = A*x_k_p;
333     P_k1_m = A*P_k_p*A.' + B*Q*B.';
334     %Save
335     est(:,1) = x_k_p;
336     err(:,:,1) = P_k_p;
337     %Run for k=1 too k=N-1
338     for k = 2:N-1
339         %Get
340         P_k_m = P_k1_m;
341         x_k_m = x_k1_m;
342         %Gain
343         K_k = P_k_m*(C.')*inv(C*P_k_m*C.' + R);
344         %Update
345         x_k_p = x_k_m + K_k*(y_n(:,k) - C*x_k_m);
346         P_k_p = (eye(2)-K_k*C)*P_k_m;
347         %Propogate
348         x_k1_m = A*x_k_p;
349         P_k1_m = A*P_k_p*A.' + B*Q*B.';
350         %Save
351         est(:,k) = x_k_p;
352         err(:,:,k) = P_k_p;
353     end
354     %End for k=N
355     %Get
356     P_k_m = P_k1_m;
357     x_k_m = x_k1_m;
358     %Gain
```

```
359     K_k = P_k_m*(C.')*inv(C*P_k_m*C.' + R);
360     %Update
361     x_k_p = x_k_m + K_k*(y_n(k) - C*x_k_m);
362     P_k_p = (eye(2)-K_k*C)*P_k_m;
363     %Save
364     est(:,N) = x_k_p;
365     err(:,:,N) = P_k_p;
366     %Return
367 end
368
369 [est_1,err_1] = kalfil2(y_n,A,B,C,Q,R, [0,0].', [0 0; 0
       0]);
370 [est_2,err_2] = kalfil2(y_n,A,B,C,Q,R, [1,1].', [0.01
       0.01; 0.01 0.01]);
371 [est_3,err_3] = kalfil2(y_n,A,B,C,Q,R, [100,100].', [1 1;
       1 1]);
372
373 %Test covergance (estimate vs true state) with each
       varying initial value
374 %Calculate the Euclidean norm between the true state and
       estimated state.
375 %Calculate the state errors
376 diff1 = x - est_1;
377 diff2 = x - est_2;
378 diff3 = x - est_3;
379 dist1 = sqrt(sum((diff1).^2));
380 dist2 = sqrt(sum((diff2).^2));
381 dist3 = sqrt(sum((diff3).^2));
382 %plot norms
383 figure;
384 plot(1:N,[dist1.' dist2.' dist3.'] ); title('Euclidean
       norm between the true state and estimated state');
       legend('est1','est2','est3');
385 xlim([1, N]);
386
387 %Compare the state errors with their respective 3thetha
       bounds computed from the covariance matrix Pk
388 %Compute the 3thetha bounds
389 x1_3std_err_1 = sqrt(squeeze(err_1(1,1,:)))*3;
390 x2_3std_err_1 = sqrt(squeeze(err_1(2,2,:)))*3;
391
392 x1_3std_err_2 = sqrt(squeeze(err_2(1,1,:)))*3;
393 x2_3std_err_2 = sqrt(squeeze(err_2(2,2,:)))*3;
394
395 x1_3std_err_3 = sqrt(squeeze(err_3(1,1,:)))*3;
```

```
x2_3std_err_3 = sqrt(squeeze(err_3(2,2,:)))*3;

%Extract the errors per state
x1_err_1 = diff1(1,:);
x2_err_1 = diff1(2,:);

x1_err_2 = diff2(1,:);
x2_err_2 = diff2(2,:);

x1_err_3 = diff3(1,:);
x2_err_3 = diff3(2,:);

%Plot
figure;
subplot(6,1,1); plot(1:N, [x1_3std_err_1, (-1)*
   x1_3std_err_1, x1_err_1.']); title('(1) x_1 error');
   legend('Upper 3std','Lower 3std','x1 error'); xlim([1, N
   ]);
subplot(6,1,2); plot(1:N, [x2_3std_err_1, (-1)*
   x2_3std_err_1, x2_err_1.']); title('(1) x_2 error');
   legend('Upper 3std','Lower 3std','x2 error'); xlim([1, N
   ]);
subplot(6,1,3); plot(1:N, [x1_3std_err_2, (-1)*
   x1_3std_err_2, x1_err_2.']); title('(2) x_1 error');
   legend('Upper 3std','Lower 3std','x1 error'); xlim([1, N
   ]);
subplot(6,1,4); plot(1:N, [x2_3std_err_2, (-1)*
   x2_3std_err_2, x2_err_2.']); title('(2) x_2 error');
   legend('Upper 3std','Lower 3std','x2 error'); xlim([1, N
   ]);
subplot(6,1,5); plot(1:N, [x1_3std_err_3, (-1)*
   x1_3std_err_3, x1_err_3.']); title('(3) x_1 error');
   legend('Upper 3std','Lower 3std','x1 error'); xlim([1, N
   ]);
subplot(6,1,6); plot(1:N, [x2_3std_err_3, (-1)*
   x2_3std_err_3, x2_err_3.']); title('(3) x_2 error');
   legend('Upper 3std','Lower 3std','x2 error'); xlim([1, N
   ]);

%% Q8

%Define system, parameters and functions
    %Initial conditions
q_0 = [0;0;0;1];          %Initial attitude (quaternion)
beta_0 = [0.1;0.1;0.1];   %Initial bias
```

```
423     %Parameters
424 sigma_i = 0.01;  %Star camera noise
425 var_i = (sigma_i^2)*eye(3);
426 R = blkdiag(var_i, var_i , var_i);
427 omega = [0;0;pi/2700];   %Angular velocity
428 r1 = [1;0;0];            %Reference basis vector
429 r2 = [0;1;0];            %Reference basis vector
430 r3 = [0;0;1];            %Reference basis vector
431 r = [r1;r2;r3];
432
433 function A = crossMatrix(a)
434     A = [  0   -a(3)  a(2);
435           a(3)   0   -a(1);
436          -a(2)  a(1)   0 ];
437 end
438
439 function pq = psi_q(quat)
440     up = quat(4)*eye(3) - crossMatrix(quat(1:3));
441     down = (-1)*quat(1:3).';
442     pq = vertcat(up,down);
443 end
444
445 function pq = xi_q(quat)
446     up = quat(4)*eye(3) + crossMatrix(quat(1:3));
447     down = (-1)*quat(1:3).';
448     pq = vertcat(up,down);
449 end
450
451 function A = q_to_matrix(quat)
452     A = (xi_q(quat).')*(psi_q(quat));
453 end
454
455 function quat = q_times(q1,q2)
456   %{
457     Implements: quat = q1 \otimes q2
458     Args:
459         q1, q2 - Two quaternions. I.e. vertical, two
                dimensional vectors
460                  of length 1.
461     Returns:
462         quat: The resulting quaternion
463   %}
464
465   quat = [psi_q(q1) q1]*q2;
466 end
```

```
467
468 function dx = rhs(t, x, omega, eta_u)
469     q = x(1:4,:);
470     omega_q = 0.5*vertcat(omega,0);
471     dq = q_times(omega_q, q);
472
473     dbeta = eta_u;
474
475     dx = vertcat(dq, dbeta);
476 end
477
478 %Scenario 1
479     %Gyro noise
480 s1_sigma_u = 1; %Bias drift
481 s1_sigma_v = 1; %Measurement noise
482 %Scenario 2
483     %Gyro noise
484 s2_sigma_u = 1; %Bias drift
485 s2_sigma_v = 2; %Measurement noise
486 %Scenario 3
487     %Gyro noise
488 s3_sigma_u = 2; %Bias drift
489 s3_sigma_v = 1; %Measurement noise
490
491 %Simulate true system
492 function [t, x, max_q_err] = simulate_sys(q_0, beta_0,
    omega, sigma_u)
493     T = 5400;
494     M = 4;
495     N = T*M+1;
496     dt = 1/M;
497     tspan = [0,dt];
498
499     t = zeros(1,N);
500     x = zeros(7,N);
501     max_q_err = 0;
502     x(:,1) = [vertcat(q_0, beta_0)];
503
504     for n = 1:N-1
505         %Get state at the start of time interval
506         x0_n = x(:,n);
507
508         %Normalize quanterion
509         q0_n = x0_n(1:4);
510         q0_n = q0_n/norm(q0_n);
```

```
511         x0_n(1:4) = q0_n;
512
513         %Replace state
514         x(:,n) = x0_n;
515
516         %Draw eta
517         eta1 = randn(1)*sigma_u;
518         eta2 = randn(1)*sigma_u;
519         eta3 = randn(1)*sigma_u;
520         eta = [eta1;eta2;eta3];
521
522         %Simulate interval
523         rhs_n = @(t,x) rhs(t, x, omega, eta);
524         [t_n, x_n] = ode45(rhs_n, tspan, x0_n);
525
526         %Log max quanterion error
527         q_norms = vecnorm(x_n(:,1:4), 2, 2);   % Euclidean
                norm of each quaternion
528         q_err = max(abs(1-q_norms));
529         max_q_err = max([max_q_err, q_err]);
530
531         %Log values at the end of the interval
532         t(n+1) = t(n) + dt;
533         x(:,n+1) = x_n(end,:).';
534
535
536     end
537 end
538
539 [s1_t, s1_x, s1_max_q_err] = simulate_sys(q_0, beta_0,
       omega, s1_sigma_u);
540 [s2_t, s2_x, s2_max_q_err] = simulate_sys(q_0, beta_0,
       omega, s2_sigma_u);
541 [s3_t, s3_x, s3_max_q_err] = simulate_sys(q_0, beta_0,
       omega, s3_sigma_u);
542
543 %Ensure they have the same true orientation
544 %I only want the biases to be different.
545 s2_x(1:4,:) = s1_x(1:4,:);
546 s3_x(1:4,:) = s1_x(1:4,:);
547
548 %Simulate measurement model
549 tol = 1e-9;
550 t_k = abs(s1_t - round(s1_t)) < tol;
551 s1_x_k = s1_x(:, t_k);
```

```
552 s2_x_k = s2_x(:, t_k);
553 s3_x_k = s3_x(:, t_k);
554
555 function bi_k = draw_body_vec(quat_k, ri, sigma_i)
556     v1 = randn(1)*sigma_i;
557     v2 = randn(1)*sigma_i;
558     v3 = randn(1)*sigma_i;
559     zi = [v1;v2;v3];
560
561     bi = q_to_matrix(quat_k)*ri + zi;
562     bi_k = bi/norm(bi);
563 end
564
565 function omega_k = draw_omega(omega, beta_k, sigma_v)
566     v1 = randn(1)*sigma_v;
567     v2 = randn(1)*sigma_v;
568     v3 = randn(1)*sigma_v;
569     eta_v = [v1;v2;v3];
570
571     omega_k = omega + beta_k + eta_v;
572 end
573
574 function s_y_k = simulate_meas(s_x_k, sigma_v, sigma_i, r1
       , r2, r3, omega)
575     J = length(s_x_k);
576     s_y_k = zeros(12, J);
577     for k = 1:J
578         x_k = s_x_k(:,k);
579         quat_k = x_k(1:4,:);
580         beta_k = x_k(5:7,:);
581
582         b1_k = draw_body_vec(quat_k, r1, sigma_i);
583         b2_k = draw_body_vec(quat_k, r2, sigma_i);
584         b3_k = draw_body_vec(quat_k, r3, sigma_i);
585
586         omega_k = draw_omega(omega, beta_k, sigma_v);
587
588         s_y_k(:,k) = vertcat(b1_k, b2_k, b3_k, omega_k);
589     end
590
591 end
592
593 s1_y_k = simulate_meas(s1_x_k, s1_sigma_v, sigma_i, r1, r2
       , r3, omega);
```

```
s2_y_k = simulate_meas(s2_x_k, s2_sigma_v, sigma_i, r1, r2, r3, omega);
s3_y_k = simulate_meas(s3_x_k, s3_sigma_v, sigma_i, r1, r2, r3, omega);

%I want them to have the same attitude measurements but different gyro
%measurements
s2_y_k(1:9,:) = s1_y_k(1:9,:);
s3_y_k(1:9,:) = s1_y_k(1:9,:);

%Init filter params
p0_a = 2; %Error covariance of Euler angles
p0_b = 2; %Error covariance of Gyro bias
p0 = diag([p0_a p0_a p0_a p0_b p0_b p0_b]);
e_q0 = [0; 0; 0.976327476659339; -0.216297615150996]; % Expected Initial attitude (quanternion)
e_beta0 = [1.020260509045736; -0.042714399068091; -0.233307253785185]; %Expected Initial gyro bias


%Implement Filter
function Hk = meas_jacobian(qk_m, r1, r2, r3)
    A = q_to_matrix(qk_m);
    Hk = [
        crossMatrix(A*r1) zeros(3,3);
        crossMatrix(A*r2) zeros(3,3);
        crossMatrix(A*r3) zeros(3,3)
    ];
end

function hk = meas_func(qk_m, r1, r2, r3)
    A = q_to_matrix(qk_m);
    hk = [A*r1; A*r2; A*r3];
end

function Kk = gain_k(pk_m, Hk, R)
    Kk = pk_m*(Hk.')*inv(Hk*pk_m*(Hk.') + R);
end

function [qk_p, betak_p, omegak_p, pk_p] = output_to_est(r, yk, qk_m, betak_m, pk_m, R)
    %Takes output at time k and updates.
    %Unpack and prepare variables
    bk_m = yk(1:9);
```

```
633     omegak_m = yk(10:12);
634     r1 = r(1:3);
635     r2 = r(4:6);
636     r3 = r(7:9);
637     %Generate matrices.
638     Hk = meas_jacobian(qk_m, r1, r2, r3);
639     hk = meas_func(qk_m, r1, r2, r3);
640     Kk = gain_k(pk_m, Hk, R);
641     %Generate error state and unpack delta alpha and beta
642     err_st = Kk*(bk_m-hk);
643     delta_alpha = err_st(1:3);
644     delta_beta = err_st(4:6);
645     %Update covariance
646     pk_p = (eye(6)-Kk*Hk)*pk_m;
647     %Update quaternion
648     qk_p = qk_m + 0.5*xi_q(qk_m)*delta_alpha;
649     qk_p = qk_p /norm(qk_p);
650     %Update beta
651     betak_p = betak_m + delta_beta;
652     %Update omega
653     omegak_p = omegak_m - betak_p;
654     %Return
655 end
656
657 function psik = psi_vector(omegak_p)
658     sclr = sin(0.5*norm(omegak_p))/norm(omegak_p);
659     psik = sclr*omegak_p;
660 end
661
662 function O = prop_q_matrix(omegak_p)
663     psik = psi_vector(omegak_p);
664     o_norm = norm(omegak_p);
665
666     O11 = cos(0.5*o_norm)*eye(3) - crossMatrix(psik);
667     O12 = psik;
668     O21 = (-1)*psik.';
669     O22 = cos(0.5*o_norm);
670
671     O = [O11 O12; O21 O22];
672 end
673
674 function gammak = prop_gamma_matrix()
675     gammak = [(-1)*eye(3) zeros(3,3); zeros(3,3) eye(3)];
676 end
677
```

```
678
679 function Qk = prop_Q_matrix(sigma_v, sigma_u)
680     var_v = sigma_v^2;
681     var_u = sigma_u^2;
682     Q11 = ( var_v + (1/3)*var_u )*eye(3);
683     Q12 = ( (1/2)*var_u )*eye(3);
684     Q21 = ( (1/2)*var_u )*eye(3);
685     Q22 = ( var_u )*eye(3);
686     Qk = [Q11 Q12; Q21 Q22];
687 end
688
689 function thethak = prop_thetha_matrix(omegak_p)
690     cmo = crossMatrix(omegak_p);
691     cmo_2 = cmo^2;
692     o_norm = norm(omegak_p);
693     o_norm_2 = o_norm^2;
694
695     thet11 = eye(3) - cmo*(sin(o_norm)/o_norm) + cmo_2*( (
            1 - cos(o_norm) ) / ( o_norm_2 ) );
696     thet12 = cmo*( ( 1 - cos(o_norm) ) / ( o_norm_2 ) ) -
           eye(3) - cmo_2*( ( o_norm - sin(o_norm) ) / ( o_norm
           ^3 ) );
697     thet21 = zeros(3,3);
698     thet22 = eye(3);
699     thethak = [thet11 thet12; thet21 thet22];
700 end
701
702
703 function [qk1_m, betak1_m, pk1_m] = prop_est(qk_p, betak_p
       , omegak_p, pk_p, sigma_v, sigma_u)
704     %Takes updated estimates and propogates
705     %Propogate beta
706     betak1_m = betak_p;
707     %Propogate quaternion
708     O = prop_q_matrix(omegak_p);
709     qk1_m = O*qk_p;
710
711     %Propogate covariance
712     gammak = prop_gamma_matrix();
713     Qk = prop_Q_matrix(sigma_v, sigma_u);
714     thethak = prop_thetha_matrix(omegak_p);
715     pk1_m = thethak*pk_p*(thethak.') + gammak*Qk*(gammak
           .');
716 end
717
```

```
718 %Simulate estimator
719 function [est_state, err_covar] = simulate_filter(r, R,
       sigma_u, sigma_v, y, p0, e_q0, e_beta0 )
720     %Takes entire meas sequence + params and returns
           estimate sequence + errors
721     N = length(y);
722     est_state = zeros(10,N); %First quaternion, then bias,
           then angular velocity
723     err_covar = zeros(6,6,N);
724
725     %Est initial
726     [qk_p, betak_p, omegak_p, pk_p] = output_to_est(r, y
           (:,1), e_q0, e_beta0, p0, R);
727     est_state(1:4,1) = qk_p;
728     est_state(5:7,1) = betak_p;
729     est_state(8:10,1) = omegak_p;
730     err_covar(:,:,1) = pk_p;
731     %Est and prop open interval
732     for k = 1:N-1
733         %Propogate
734         qk_p = est_state(1:4,k);
735         betak_p = est_state(5:7,k);
736         omegak_p = est_state(8:10,k);
737         pk_p = err_covar(:,:,k);
738         [qk1_m, betak1_m, pk1_m] = prop_est(qk_p, betak_p,
               omegak_p, pk_p, sigma_v, sigma_u);
739
740         %Update
741         yk1 = y(:,k+1);
742         [qk1_p, betak1_p, omegak1_p, pk1_p] =
              output_to_est(r, yk1, qk1_m, betak1_m, pk1_m, R)
              ;
743
744         est_state(1:4,k+1) = qk1_p;
745         est_state(5:7,k+1) = betak1_p;
746         est_state(8:10,k+1) = omegak1_p;
747         err_covar(:,:,k+1) = pk1_p;
748     end
749
750     %Return
751 end
752
753 [s1_est_state, s1_err_covar] = simulate_filter(r, R,
       s1_sigma_u, s1_sigma_v, s1_y_k, p0, e_q0, e_beta0);
```

```
754 [s2_est_state, s2_err_covar] = simulate_filter(r, R,
       s2_sigma_u, s2_sigma_v, s2_y_k, p0, e_q0, e_beta0);
755 [s3_est_state, s3_err_covar] = simulate_filter(r, R,
       s3_sigma_u, s3_sigma_v, s3_y_k, p0, e_q0, e_beta0);
756
757 %Assess performance
758 function [ate, bie, ave] = calc_error(true_x, true_omega,
       est_x)
759     N = length(true_x);
760
761     ate = zeros(1,N);
762     bie = zeros(1,N);
763     ave = zeros(1,N);
764     for k = 1:N
765         %Attitude error (Forbenius norm between attitude
              matrices)
766         truA = q_to_matrix(true_x(1:4,k));
767         estA = q_to_matrix(est_x(1:4,k));
768         ate(k) = norm(truA-estA, 'fro');
769         %Bias error (Euclidean norm between bias vectors)
770         truBeta = true_x(5:7,k);
771         estBeta = est_x(5:7,k);
772         bie(k) = norm(truBeta-estBeta);
773         %Angular velocity error (Euclidean norm between
              velocity vectors)
774         estOmega = est_x(8:10, k);
775         ave(k) = norm(true_omega-estOmega);
776     end
777 end
778
779 [s1_ate, s1_bie, s1_ave] = calc_error(s1_x_k, omega,
       s1_est_state);
780 [s2_ate, s2_bie, s2_ave] = calc_error(s2_x_k, omega,
       s2_est_state);
781 [s3_ate, s3_bie, s3_ave] = calc_error(s3_x_k, omega,
       s3_est_state);
782
783 function [avg_ea, avg_bia] = calc_avg_var(err_covar)
784     N = length(err_covar);
785
786     avg_ea = zeros(1,N);
787     avg_bia = zeros(1,N);
788     for k = 1:N
789         elem = diag(err_covar(:,:,k));
790         ea = elem(1:3);
```

```
791             bia = elem(4:6);
792             %Average variance of euler angles
793             avg_ea(k) = mean(ea);
794             %Average variance of Bias
795             avg_bia(k) = mean(bia);
796         end
797     end
798     [s1_avg_ea, s1_avg_bia] = calc_avg_var(s1_err_covar);
799     [s2_avg_ea, s2_avg_bia] = calc_avg_var(s2_err_covar);
800     [s3_avg_ea, s3_avg_bia] = calc_avg_var(s3_err_covar);
801
802     %plot results
803
804     %1:
805     figure;
806     N = length(s1_x_k);
807     %Attitude error
808     subplot(3,1,1); plot(1:N, [s1_ate.',s2_ate.',s3_ate.']);
            title('Attitude error (Forbenius norm)'); legend('
            Scenario 1','Scenario 2','Scenario 3'); xlim([1, N]);
809     %Bias error
810     subplot(3,1,2); plot(1:N, [s1_bie.',s2_bie.',s3_bie.']);
            title('Gyro Bias error (Euclidean norm)'); legend('
            Scenario 1','Scenario 2','Scenario 3'); xlim([1, N]);
811     %Angular velocity error
812     subplot(3,1,3); plot(1:N, [s1_ave.',s2_ave.',s3_ave.']);
            title('Angular velocity error (Euclidean norm)'); legend
            ('Scenario 1','Scenario 2','Scenario 3'); xlim([1, N]);
813
814     %2:
815     figure;
816     %Avg euler angle variance
817     subplot(2,1,1); plot(1:N, [s1_avg_ea.',s2_avg_ea.',
            s3_avg_ea.']); title('Average Variance of Euler Angle
            Residuals'); legend('Scenario 1','Scenario 2','Scenario
            3'); xlim([1, N]);
818     %Avg bias variance
819     subplot(2,1,2); plot(1:N, [s1_avg_bia.',s2_avg_bia.',
            s3_avg_bia.']); title('Average Variance of Gyro Bias
            Residuals'); legend('Scenario 1','Scenario 2','Scenario
            3'); xlim([1, N]);
```